The Agentic Avengers

Background and Motivation

Many communities lack timely and reliable digital safety intelligence due to fragmented reporting systems and delayed response mechanisms. As phishing, impersonation, and online scams become more sophisticated, individuals and institutions face increasing financial and security risks.

Existing tools are often reactive, non-transparent in how risk decisions are made, and limited in community collaboration. Users are typically warned only after interacting with malicious content, and they are rarely shown why a site is dangerous.

To address these gaps, Antiphish+ introduces a transparent and community-driven phishing detection platform supported by an integrated AI-powered browser extension. Unlike passive detection systems, the extension provides real-time protection by actively blocking high-risk pages before users enter sensitive information.

Project Statement (Antiphish)

Antiphish+ is a community-driven anti-phishing platform designed to improve digital safety through transparent and explainable URL risk detection. The system allows verified users to log in, analyze website links, and receive a percentage-based phishing risk score. Unlike black-box systems, Antiphish+ provides clear explanations for each risk factor, such as insecure protocols, suspicious domains, or phishing-related keywords. Users can report flagged suspicious links to a shared Community Intelligence Feed, strengthening collective threat awareness while preventing misuse by disabling reports for safe links. The platform also includes Smart Safety Tips to educate users on **identifying phishing patterns independently,** along with a **downloadable AI-powered browser extension** that provides real-time protection during live browsing. Together, these components create a scalable, privacy-conscious digital safety solution.

Objective

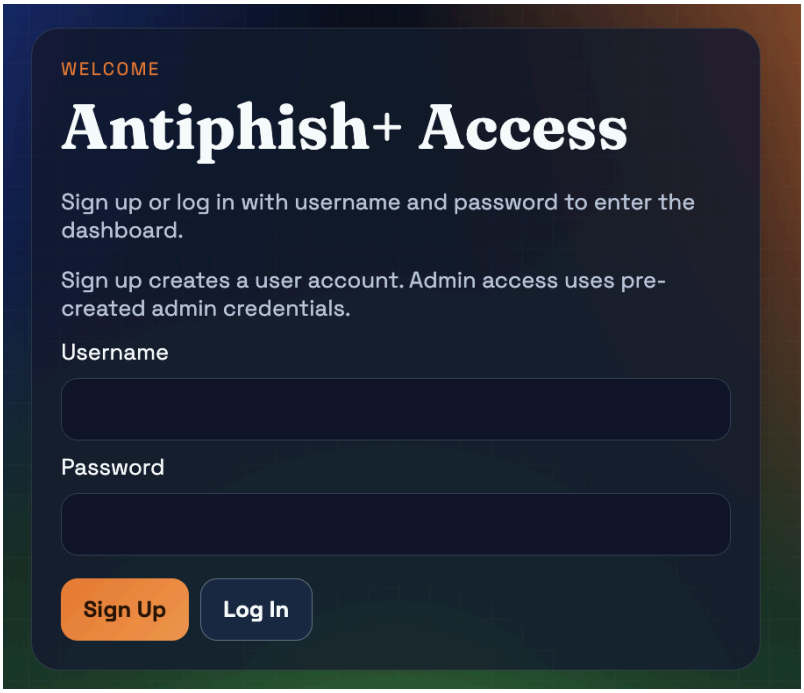
- Deliver fast, explainable URL phishing detection with clear risk scores and actionable guidance.
- Preserve privacy by analyzing URL patterns without fetching webpage content (SSRF-safe design).

- Support verified community reporting with a searchable threat intelligence feed.
- Maintain stable API structures for consistent frontend and extension integration.
- Provide real-time browser protection through an integrated detection extension.

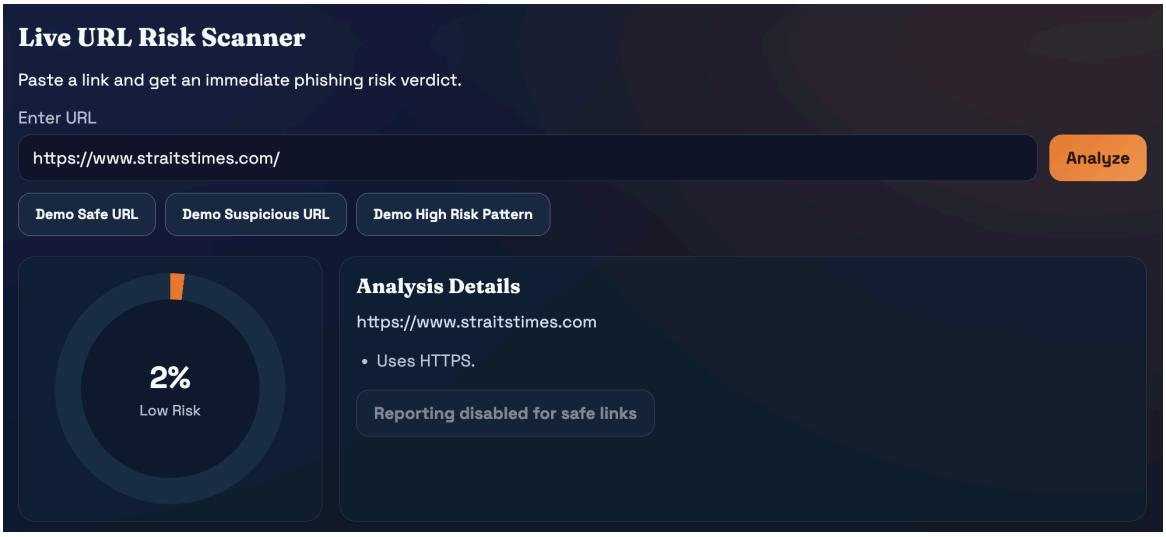
System Overview and Platform Features

Antiphish+ is a privacy-conscious, community-driven anti-phishing platform that combines URL risk detection, explainable analysis, community reporting, and real-time browser protection.

The system is designed around five core components: user authentication, live URL scanning, explainable AI analysis, community threat intelligence, and browser-level integration.

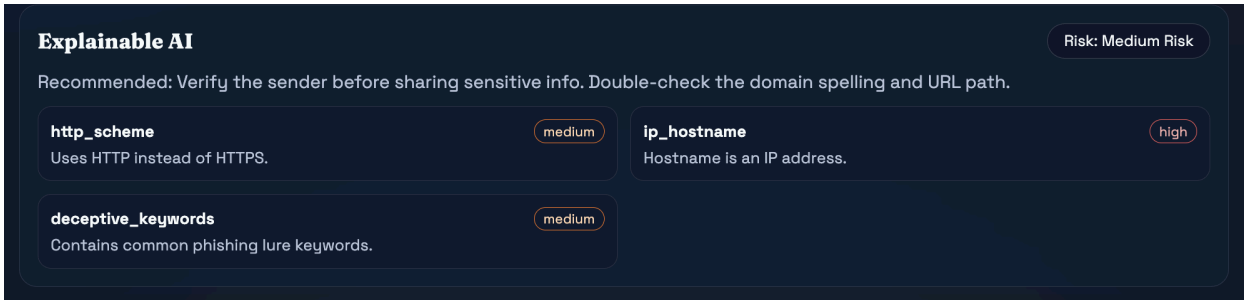


Users must create an account and log in before accessing the dashboard. This ensures that all phishing reports are tied to verified users rather than anonymous sources. By preventing anonymous submissions, the system strengthens accountability and maintains the integrity of the community intelligence feed.



After logging in, users can paste any website link into the Live URL Risk Scanner. The backend analyzes the URL using structured security heuristics and generates a percentage-based phishing risk score categorized as Low, Medium, or High Risk.

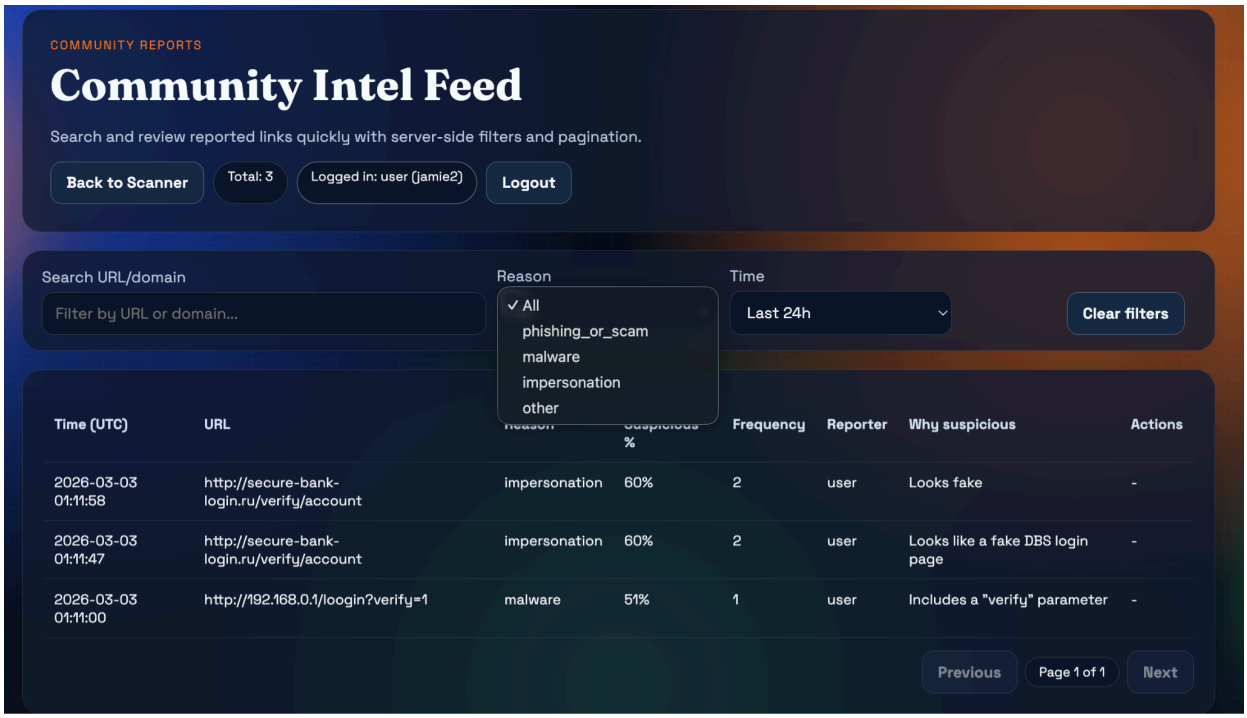
If a link is assessed as safe, reporting is disabled to prevent misuse. If flagged as suspicious, users can submit the link to the Community Intelligence Feed.



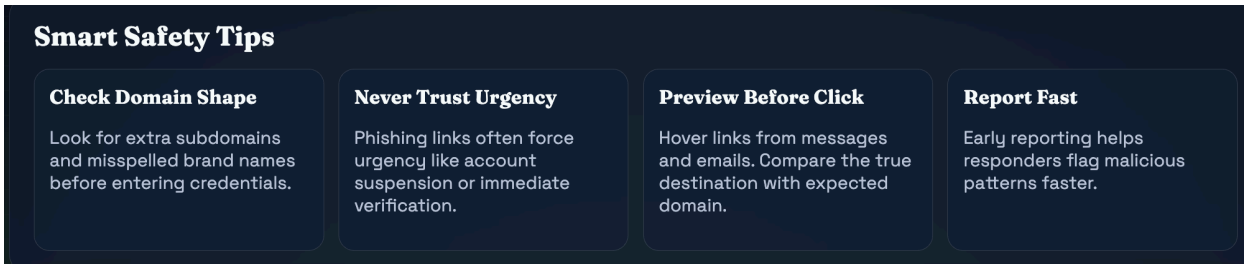
Instead of providing a black-box verdict, Antiphish+ explains the reasoning behind the assigned risk score. Detected factors may include:

- Insecure HTTP usage
- Suspicious IP-based hostnames
- High-risk top-level domains
- Common phishing lure keywords

Each factor is categorized by severity (e.g., medium, high), enabling users to understand not only the result but the underlying reasoning. This transparency builds user trust and improves digital literacy.



All suspicious links submitted by users are aggregated into a shared Community Intelligence Feed.
This creates a collaborative threat intelligence layer. As more users report suspicious URLs, the system becomes more comprehensive and reliable.



To promote long-term digital safety, the platform includes practical Smart Safety Tips. These guidelines help users independently recognize phishing patterns, such as:

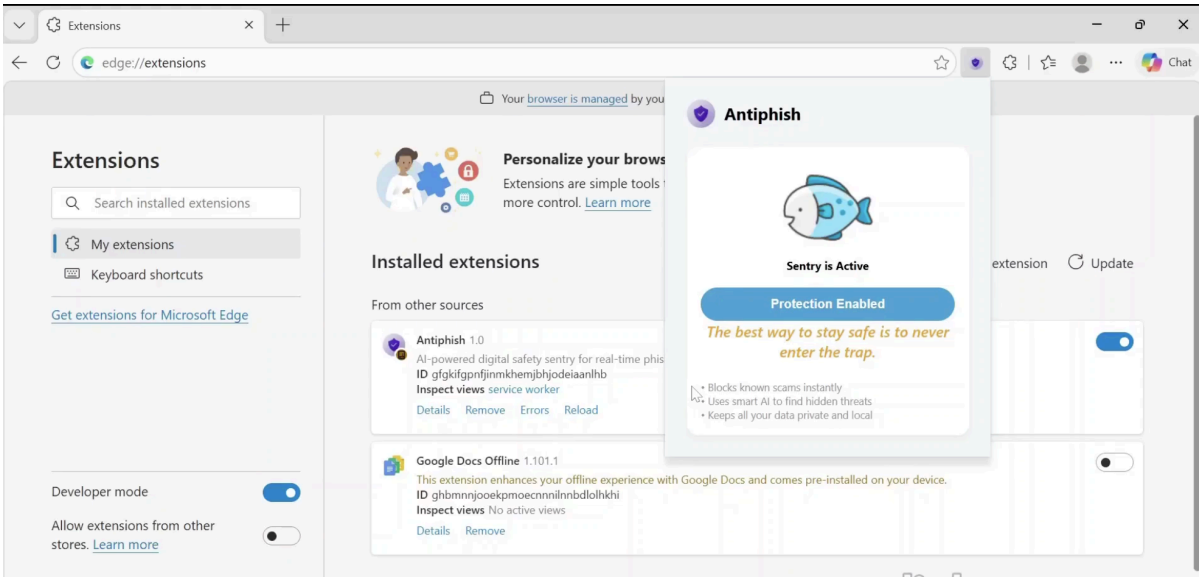
- Checking domain spelling
- Avoiding urgency-based messages
- Previewing links before clicking
- Reporting suspicious sites early

This educational layer reinforces human awareness alongside automated detection.

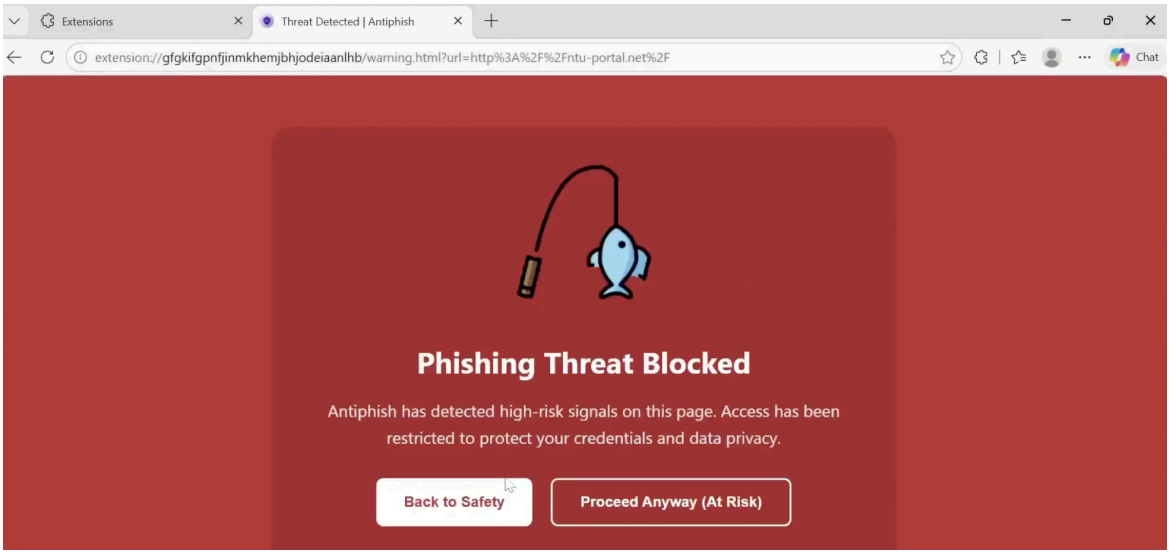
Browser Extension Setup (Edge)

1. Download the extension zip using the button above.
2. Extract the zip file to a folder on your computer.
3. Navigate to `edge://extensions`.
4. Enable **Developer mode**.
5. Click **Load unpacked** and select the extracted extension folder.

Users can download and install the Antiphish+ browser extension directly from the platform. The extension connects to the backend API and analyzes URLs in real time during live browsing. This enables proactive protection beyond manual URL scanning, extending the system's capabilities into real-world usage environments.



Our browser extension includes a simple enable and disable toggle. This allows users to temporarily turn off protection if they encounter a false positive. Instead of forcing automatic blocking, we prioritize user control and flexibility while still maintaining real-time phishing detection.



When the browser extension detects high-risk phishing signals, it automatically intercepts the page request and displays a warning screen. Access to the suspicious site is restricted to protect users from credential theft or data compromise.

The system provides two options:

- **Back to Safety** – Redirects the user away from the dangerous page.
- **Proceed Anyway (At Risk)** – Allows the user to override the block in case of a false positive.

This design ensures strong proactive protection while maintaining user control. By combining automated blocking with human decision-making, the system follows a responsible security-by-design approach.

Technology Stack

This project is implemented using a modern full-stack web architecture with cloud deployment and browser-based intelligence components.

FrontEnd

Framework & Tools

- HTML / CSS / JavaScript
- Vite (development tooling)
- Fetch API for backend communication

Deployment

- Vercel (serverless frontend hosting)

Frontend Responsibilities

- User interface for submitting URLs for phishing analysis
- Display of risk scores and explainable results
- Community Intelligence Feed for reported URLs
- Integration with backend APIs via HTTPS

Frontend URL

- https://team-agentic-avengers-antiphish.vercel.app

Backend

Language & Framework

- Python 3.12
- FastAPI (high-performance API framework)
- Uvicorn (ASGI server)

Key Libraries

- pydantic – request/response validation
- fastapi – API routing and documentation
- python-dotenv – environment variable handling
- requests – external calls if needed

Backend Responsibilities

- URL phishing risk analysis
- Explainable risk factor generation
- Community report handling
- API documentation via Swagger UI

Backend Deployment

- Render (Web Service)

API Documentation

- https://team-agentic-avengers-antiphish.onrender.com/docs

Browser Extension

Technologies

- JavaScript
- Chrome Extension APIs
- HTML / CSS

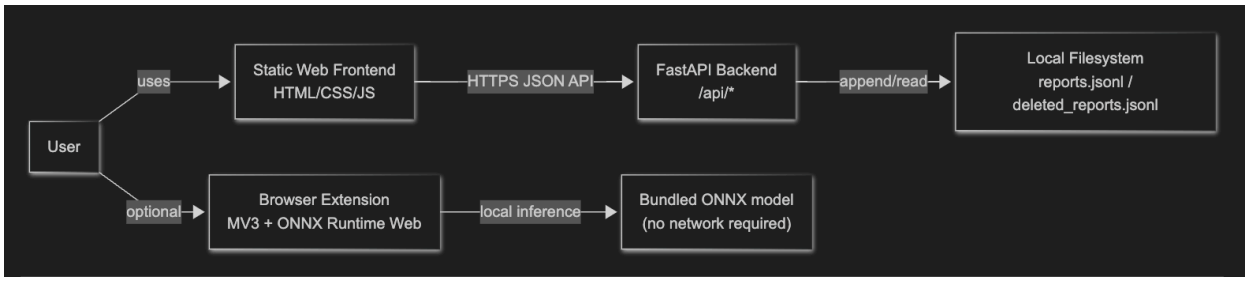
Responsibilities

- Real-time phishing detection while browsing
- URL extraction from active tabs
- Direct communication with backend API
- One-click reporting to the Community Intelligence Feed

Source Code

- extension/ directory in the repository

System Architecture



Overall system architecture flow diagram

Edge inference pipeline diagram

User Interface Considerations

Theme Colours

Antiphish+ uses a modern, security-centric color system built around deep navy tones and high-contrast accents. Ink Black (#0B1220) and layered blues (#111B2F, #182943) define the primary background, creating depth and reinforcing a cybersecurity look and feel.

Accent colors are used intentionally to signal actions and risk states. Atomic Tangerine (#F97136) highlights primary CTAs like the Analyze button, while Jade Green (#22C55E) represents safe or low-risk outcomes. Lighter blues (#B8C7DB, #BECCDE) support secondary elements and typography, preserving hierarchy and readability without visually overloading the interface.

Typography

The Antiphish+ interface uses a carefully selected font system to balance clarity, modern aesthetics, and readability across both the web dashboard and browser extension.

For the main web application, Space Grotesk (sans-serif) is used as the primary UI font. Its clean geometric structure enhances readability within the dark-themed interface while maintaining a modern cybersecurity aesthetic.

To create visual emphasis and hierarchy, Fraunces (serif) is used selectively for headings and highlighted titles such as section headers and hero text. The contrast between the serif display font and the primary sans-serif UI font reinforces structure and improves scannability.

For the browser extension interfaces (popup and warning pages), a system-based sans-serif stack centered around Segoe UI is used. This ensures native rendering, fast loading, and consistent appearance across Windows and Chromium-based environments. Using system fonts in the extension also improves performance and visual integration with the browser environment.

Heading â€“ Fraunces

Subheading â€“ Fraunces

Text XL â€“ Space Grotesk

Text MD â€“ Space Grotesk

Text SM â€“ Space Grotesk

Accessibilty

1. Antiphish+ is designed to be accessible and usable for a wide range of users, including those with visual or motor impairments. The initial release will support English as the primary language across the web app and browser extension, with future iterations considering localization based on user needs.

2. The interface uses a high-contrast, dark-themed color palette and legible typography (Space Grotesk for body text and Fraunces for headings) to support readability in low-light and high-glare environments.

3. Core flows such as entering a URL, viewing analysis results, and interacting with extension warnings are designed to be fully keyboard navigable, with clear focus states and a logical tab order. Interactive controls including buttons, links, and toggles use native HTML elements wherever possible, or appropriate ARIA attributes when necessary, to ensure compatibility with screen readers and assistive technologies.

4. Visual indicators in the browser extension (safe, suspicious, or high risk) are accompanied by clear textual descriptions so that users relying on assistive

technologies can understand the security state without needing to interpret colors or icons alone.

Implementation Details

1. System Architecture Overview

Antiphish+ is implemented as a three-tier system consisting of:

1. Frontend (Vercel-hosted web application)
2. Backend API (FastAPI deployed on Render)
3. Browser Extension (Chrome Extension API)

The frontend and extension communicate with the backend through structured RESTful API calls. The backend processes URL inputs, computes risk scores, and returns standardized responses used by both the dashboard and extension.

This modular design ensures scalability, separation of concerns, and consistent system behavior across platforms.

2. Frontend Implementation

The frontend is a web-based dashboard deployed on Vercel (live at https://team-agentic-avengers-antiphish.vercel.app) that provides an accessible interface for phishing analysis and community intelligence. Users can sign up and log in, then submit a URL through the Live URL Risk Scanner.

The frontend sends structured requests to the backend API and renders the results, including a percentage-based riskScore and a human-readable risk label (Low Risk / Medium Risk / High Risk), plus an explainable breakdown of detected signals and recommendedActions.

Report submission is controlled by the backend verdict: reporting is disabled for URLs classified as low risk and enabled only for medium or high-risk links, with authentication required. The Community Intelligence Feed retrieves reported entries from the backend and displays them in a searchable, filterable, and paginated list.

The page also includes Smart Safety Tips and provides an extension download link with installation steps for Chromium-based browsers. The UI relies on stable backend response fields for consistent rendering across updates.

3. Backend Implementation

The backend is implemented using Python 3.12 and FastAPI, providing RESTful API endpoints for URL analysis and community reporting. When a URL is submitted, the system performs validation and normalization, including scheme (HTTP/HTTPS) and hostname checks. Malformed inputs are rejected early. The system does not fetch or execute webpage content, ensuring an SSRF-safe and privacy-preserving design.

After validation, structured heuristic checks are applied to detect phishing-related signals such as insecure protocols, suspicious domains, and phishing keywords. Each signal contributes to a cumulative integer risk score (0–100), which is mapped to a categorical label (low, medium, high) based on predefined thresholds.

The API returns a stable JSON response containing url, normalizedUrl, timestamp, riskScore, riskLabel, signals, and recommendedActions, ensuring seamless integration with both the frontend and browser extension. Suspicious reports submitted by authenticated users are persisted to a lightweight reports.jsonl file. The backend is deployed on Render and configured using environment variables for secure CORS and runtime management.

4. Browser Extension Implementation

The browser extension is implemented as a client-side security layer that operates within the Chrome Extension Manifest V3 (MV3) environment. The architecture centers on a non-persistent, event-driven Service Worker (background.js) which serves as the extension's primary event handler. Because MV3 mandates that all executable code must be locally bundled to prevent the execution of remotely hosted scripts, the extension includes all necessary WebAssembly (WASM) binaries and the machine learning model within its package. The detection is powered by a Random Forest classifier, which was trained with scikit-learn and exported to the Open Neural Network Exchange (ONNX) format for cross-platform compatibility.

The detection logic is triggered whenever a user navigates to a new URL or a tab is updated. The system first checks the URL against local Blacklists and Whitelists to provide instantaneous responses for known threats or trusted domains. If the URL is unknown, the Service Worker initiates a real-time Feature Extraction process, transforming the URL string into a 12-dimensional numerical vector. These features include structural signals such as URL length, the presence of a robots.txt and domain age. This vector is then processed by the bundled ONNX Runtime using a WebAssembly backend.

To optimize performance and minimize redundant calculations, the extension implements a local Inference Cache. Once a URL is analyzed, its risk status is stored in memory, allowing for immediate blocking or allowing during subsequent visits within the same session. User preferences and whitelisted "proceeded" URLs are persisted using chrome.storage.local, ensuring that the extension's state remains consistent even after the Service Worker hibernates and restarts. If a site is flagged as phishing, the Service Worker intercepts the navigation and redirects the tab to a local Warning Page (warning.html), which provides the user with an explanation of the risk and the option to either return to safety or manually override the block, thereby maintaining a balance between automated protection and user agency.

Testing and Validation

The system underwent functional, integration, and deployment-level testing to ensure reliability and correctness. URL analysis was validated using safe, suspicious, and malformed inputs to confirm accurate risk scoring, signal detection, and recommended actions. API responses were verified for stable JSON structure consistency across multiple test cases to ensure seamless frontend and browser extension integration.

Frontend testing confirmed correct rendering of percentage-based risk scores, reporting restrictions for safe links, and proper updates to the Community Intelligence Feed.

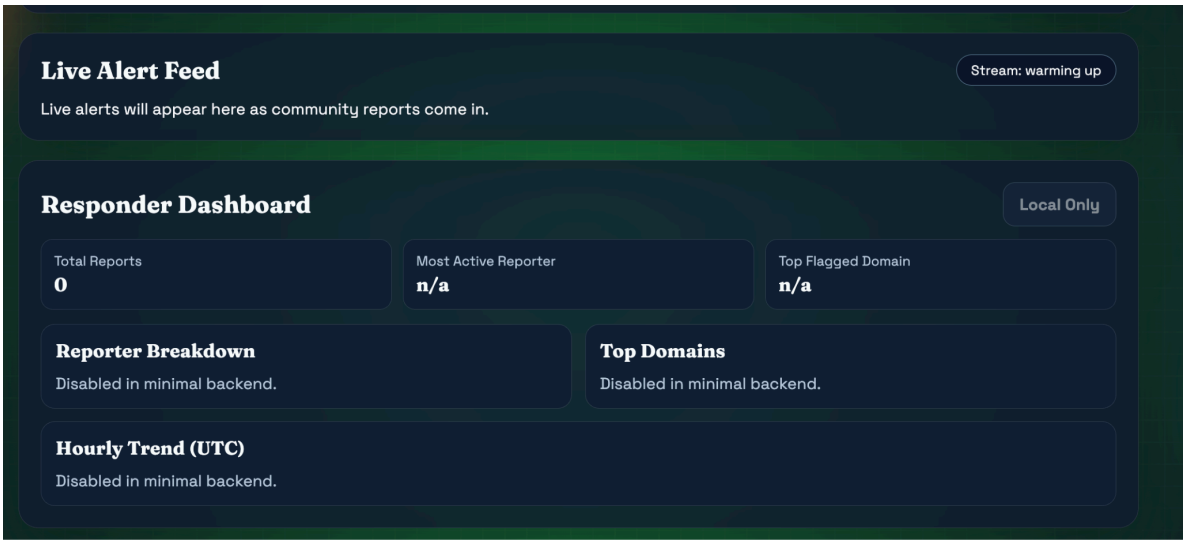
The browser extension was tested for real-time detection, automatic blocking of high-risk URLs, manual override functionality, and toggle-based false positive handling.

Edge cases including empty input, invalid URLs, duplicate submissions, and abnormal URL lengths were tested to confirm graceful error handling. Deployment validation ensured stable connectivity between Vercel (frontend), Render (backend), and the Chrome extension environment.

Test Case	Input	Expected Output	Actual Output	Pass
Abnormal URL Lengths	http://example.com/login/verify/account/update/session/token/secure/authentication/check/aaaa

HTTP Instead of HTTPS	http://secure-bank-login.ru/verify/account	Medium risk, http_scheme signal triggered	Risk score increased, http_scheme detected	✔
IP-Based Hostname	http://192.168.0.1/login?verify=1	Medium or High risk, ip_hostname signal triggered	Risk score increased, ip_hostname detected	✔
Phishing Keyword Detection	http://secure-login-verify.com/account	Elevated risk, deceptive_keywords signal	Risk score increased, deceptive_keywords detected	✔
Legitimate HTTPS Website	https://www.nytimes.com	Low risk returned, report button disabled	Low risk returned, report button disabled	✔
Reporting Restriction	https://www.straitstimes.com/	Report button disabled	Report button disabled	✔
Automatic Blocking (Extension)	ntu-portal.net	Warning page displayed, access restricted	Phishing Threat Blocked screen shown	✔
Manual Override (Extension)	ntu-portal.net	User allowed to continue	User was able to continue	✔
Toggle False Positive Handling	Toggle protection OFF	No blocking triggered	Extension did not intercept page	✔

Limitations and Future Work



1. Local Aggregation
    - The Live Alert Feed currently operates in a simplified mode and does not use a real-time streaming or database-backed aggregation system. Reports are stored in a local JSONL file rather than a scalable database.
    - Future work includes replacing the current JSONL-based storage with a production-grade database to support higher report volumes, structured querying, and scalable analytics.
2. Heuristic-Based Detection
    - The Explainable AI module is rule-based and heuristic-driven rather than powered by a trained machine learning model. While this ensures transparency and deterministic outputs, it may not detect novel or highly sophisticated phishing patterns.
    - Future work includes enhancing the Explainable AI module by incorporating supervised machine learning models trained on labeled phishing datasets. This would improve detection accuracy for emerging and sophisticated attack patterns while preserving interpretability through feature attribution and signal transparency.

3. Browser extension can be made cross-platform compatible (currently only for chrome and edge, no firefox etc)
    - Zero Day vulnerability may go undetected, Static machine learning models and blacklists often fail to detect brand-new phishing websites (zero-hour attacks) because they haven't yet been added to databases or don't match previously learned patterns.

Conclusion

Antiphish+ delivers a transparent, community-driven approach to phishing detection by combining explainable URL risk analysis, verified reporting, and real-time browser protection. Users receive not only a risk score, but a clear breakdown of why a link is suspicious, building trust and improving digital awareness.

The highlight of the project is the AI-powered browser extension, which provides instant phishing detection during live browsing and actively blocks high-risk websites while allowing user override for flexibility. By integrating the dashboard, backend API, and extension into a unified ecosystem, Antiphish+ demonstrates how scalable, explainable, and user-centered cybersecurity solutions can strengthen digital safety in real-world environments.